DD2358 Project Report

Computational Astrophysics - Smoothed-Particle Hydrodynamics
Simulation of Toy Star

Phoebe Schwartz - phoebes@kth.se
Carlotta Hölzle - csholzle@kth.se
Rose Maresh - maresh@kth.se
Christiane Kobalt - kobalt@kth.se

Expected grade: A

Introduction

Smoothed-Particle Hydrodynamics Simulation of Toy Star code written by Philip Mocz, simulates the structure of a star using Smoothed-Particle Hydrodynamics (SPH), a computational method commonly used to model fluid dynamics and gravitational systems. The SPH method is mainly used in astrophysical simulations but can be used in other fluid movement problems with complex borders. By removing the bottlenecks in simulation code, has more potential for simulating real world applications in a shorter amount of time. This code begins with randomly initialized particle positions and zero velocities, along with parameters like the star's mass, radius, and a polytrophic index that governs the pressure-density relationship. The system evolves over time as particles interact under forces like pressure gradients, viscosity, and an external potential, gradually relaxing into a configuration of equilibrium where internal pressures balance gravitational forces. This final state represents a stable stellar structure, as it achieves hydrostatic equilibrium, mimicking the balance observed in real stars. The goal of this project is to characterize the performance of this code and optimize it with several techniques we have learned in class. We will also assess and analyze the impact of the optimization techniques on the program's performance. Optimizing this simulation could enable more complex and larger-scale projects, leading to a deeper scientific understanding of our physical world.

Methodology

The Smoothed-Particle Hydrodynamics simulation of Toy Star program is 254 lines long with 7 functions to investigate. We split up our work into each of the 4 modules presented in class. Our code includes the original program, our profiling results, our optimizations, and performance measurement comparisons in python notebooks. Everything we did can be seen in our Github Repository.

To begin this project, we each analyzed the original code by running the profiling tests that we utilized earlier in the course. Since all team members have a different hardware we ran the profiling on each to see any major differences across systems. When later comparing optimization results we calculate the percentage gained in processing speed or memory usage depending on the hardware, this enables us to compare across optimizations without the need to run all optimizations on only one hardware.

Rose	11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz 2.30 GHz
Carlotta	Apple Silicon M1 Chip 8 Core
Chris	AMD Ryzen 5 5600U with Radeon Graphics @ 12x 2.301GHz and Google Colab
Phoebe	Intel(R) Core(TM) Ultra 7 155H [Cores 16] [Logical processors 22]

Module 1: Profiling

To identify the most computationally expensive methods in the original code, we analyzed the proportion of total execution time spent directly within each method. For this we profiled the code using the built-in Python `cProfile` module, executed with: `python -m cProfile -s cumulative sph.py`.

To begin, we first excluded plotting from the original code since we wanted to focus on precisely measuring the execution time of the computations. Once we modified the code, we profiled for $N = 400$, $N = 1000$ and $N = 10000$, which are different numbers of initial particles. We extracted the self-time (`tottime`) of each method and computed its percentage contribution to the total execution time. The results of this analysis are summarized in Table 1.

Method	**Percentage of Total Execution Time**
`getAcc`	∼10%
`gradW`	∼21%
`getPairwiseSeparations`	∼20%
`getDensity`	∼2.5%
`W`	∼34%

Table 1: Profiling results: Percentage of cumulative execution time spent within each method.

These results of our profiling indicate that the most computationally demanding methods are `W` (34%), `gradW` (21%), and `getPairwiseSeparations` (20%), highlighting our targets for optimization.

Hypothesis for CPU Optimization

Based on the profiling results and code structure analysis, we derived the following key hypotheses to improve computational efficiency:

1. Prioritizing the Right Optimizations Although `getAcc` accounts for only ∼10% of execution time, it frequently calls `getPairwiseSeparations`, `gradW`, and `getDensity`. Also, while `getDensity` itself accounts for only ∼2.5% of execution time, its reliance on `W` makes it indirectly expensive. Line-by-line profiling of `getAcc` and `getDensity` will help to identify redundant computations and give a better understanding of expensive function calls.

2. Vectorization of Expensive Operations The function `W` (∼34%) applies a Gaussian kernel element-wise over large NumPy arrays, making it a strong candidate for optimization using vectorization and JIT compilation. Similarly, `gradW` (∼21%) performs element-wise operations on `dx, dy, dz`, where redundant calculations, such as computing `r = np.sqrt(x**2 + y**2 + z**2)` multiple times, could be eliminated. An idea would be to leverage NumPy's broadcasting capabilities or NumExpr to reduce execution time.

3. Reducing Pairwise Computations The function `getPairwiseSeparations` (∼20%) constructs pairwise distance matrices, which scale poorly with $\mathcal{O}(N^2)$ complexity. Optimizations such as introducing parallelizing pairwise computations via multiprocessing or optimizing large matrix operations using NumPy's `einsum` function could further reduce computational overhead.

Optimizations Implemented

We decided to explore and implement optimization strategies from each module taught in class.

Module 1	**Module 2**	**Module 3**	**Module 4**
Profiling	Execution Time	Cython	Multiprocessing
	Memory	GPU	Dask

Performance Improvements

The performance impact of each implemented optimization technique was systematically assessed using the same profiling methodologies as described earlier. In the subsequent results section, we provide a comparative analysis of each method, highlighting the achieved performance gains. Due to variations in hardware architectures across our testing environments, we standardized our comparison by reporting relative improvements as percentages rather than absolute execution times.

Testing

We used `pytest` test to ensure that our optimizations resulted in the same values as the original code. These tests are part of each optimization module and can be found in our Github Repository.

Results

In this part of the report we highlight the results of our different optimization approaches.

Module 2: Processing Speed

We hypothesized that by optimizing critical functions through improved data layouts and effective vectorization we can significantly enhance computational efficiency. Profiling revealed that the primary bottlenecks, namely the functions getPairwiseSeperations, W, and gradW, contributed substantially to the total CPU runtime. To address these inefficiencies, we employed optimization strategies introduced in Module 2, which are explained in more detail below.

Optimizing `getPairwiseSeparations`: Approaches and Performance

To reduce the computational cost of `getPairwiseSeparations`, we tested several vectorization strategies, evaluating their performance both as a standalone function and when integrated within the `getDensity` function. The tested methods are as follows:

- Original (Baseline): The initial implementation using explicit NumPy operations to compute pairwise separations.
- NumPy Optimization: Utilized `np.subtract.outer` for vectorized subtraction, slightly improving memory efficiency.

- NumExpr Optimization: Applied NumExpr to parallelize computations and optimize large array operations, performance gains for small N were limited.
- Fully Vectorized: Used NumPy advanced indexing for full vectorization, but excessive memory usage caused significant slowdowns.
- In-Place Computation: Optimized memory handling by modifying arrays in place, achieving the best performance improvement.
- Rolling Window: Cached previously computed separations to minimize redundant calculations, yielding marginal improvements.
- Block-wise Computation: Split computations into smaller blocks to improve cache locality, though performance gains were inconsistent.
- Parallel Processing: Used multiprocessing to distribute computation across CPU cores, but overhead from inter-process communication resulted in a substantial slowdown.

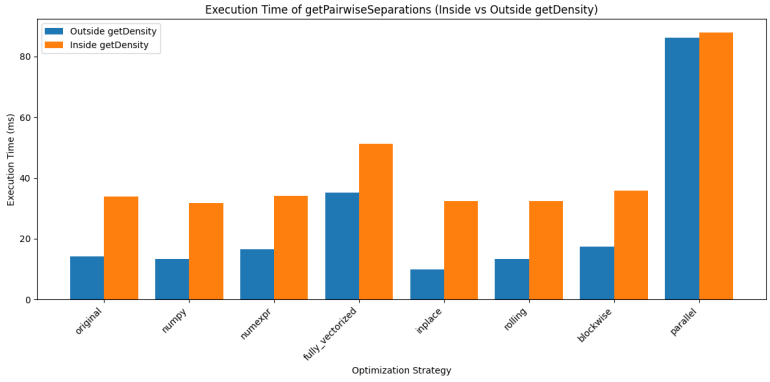


Figure 1: Comparison of different `getPairwiseSeparation` implementations

Performance Analysis Among the tested strategies, the in-place computation approach provided the most significant performance gain, outperforming the baseline as can be seen in 1. Fully vectorized and parallel methods were the least efficient due to excessive memory allocation and communication overhead, respectively. The NumPy and rolling window optimizations provided minor improvements, while block-wise and NumExpr approaches showed mixed results, depending on the number of particles (N). These findings suggest that optimizing memory access patterns is more effective than increasing computational parallelism for this function.

Optimizing the Kernel Function `W`

When targeting the inefficiencies in the W function, we noticed that the kernel is evaluated for all particles and not only the contributing pairs. To reduce computational overhead in the Gaussian smoothing kernel, we implemented an optimized version which sets the values of non-contributing particles preemptively to zero. Furthermore, we explicitly convert the smoothing length `h` to a NumPy array (`np.asarray(h)`) to enhance compatibility with NumPy operations and avoid unnecessary type conversions.

These optimizations led to a notable decrease in execution time, reducing the computational cost of density evaluation from 74.1% to 64.6% of total execution time. This demonstrates that limiting kernel evaluations to relevant neighbors effectively improves performance while preserving accuracy.

By fusing the optimizations of the getPairwiseSeparations function and the kernel function W, we reduced the execution time of getDensity by 59.9%.

Optimizing the Gradient Computation `gradW`

After optimizing `getPairwiseSeparations` and `W`, we focused on optimizing `gradW`, as it remained computationally expensive. The gradient computation involves element-wise operations on large matrices, making it a promising target for optimization using vectorization.

Figure 2 shows a comparison of execution times for different optimization strategies. The results indicate that while in-place and float32-based optimizations provided marginal improvements for small problem sizes, they failed to deliver meaningful speedups for larger particle numbers N. In contrast, NumExpr, despite performing poorly for small matrices, due to its higher overhead, became the fastest approach for larger problem sizes, highlighting the advantages of low-level parallel execution when working with large data sets.

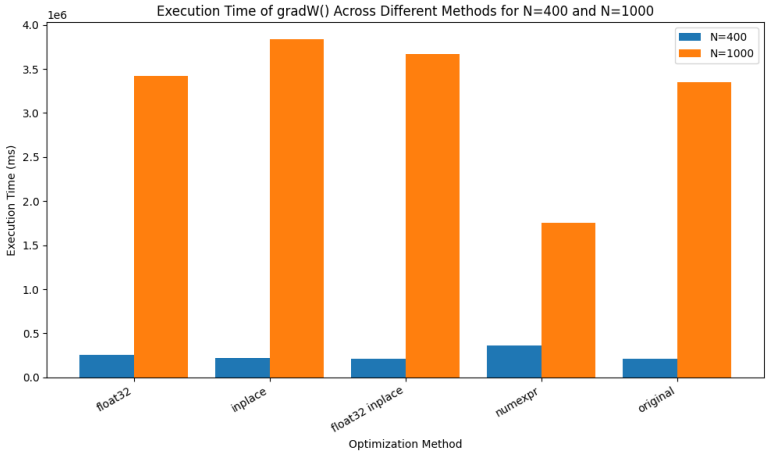


Figure 2: Comparison of different `gradW` implementations

Evaluating `memoryview` for Optimization

Because `memoryview` allows direct access to memory buffers and reduces implicit copying, we hypothesized that it could improve performance in `getPairwiseSeparations`, `W`, and `gradW`, where large NumPy arrays are frequently allocated and accessed.

Figure 3 compares execution times across different optimization stages. The results for $N = 1000$ show that using `memoryview` alone led to a 26.5% speedup, confirming an increase in efficiency. However, when combined with all optimizations, execution time increased compared to the fully optimized version without it. This suggests potential conflicts with NumPy's internal optimizations, and the non-additivity of optimization approaches.

For small N, `memoryview` alone showed no significant benefits, indicating that memory access was not the primary bottleneck. While `memoryview` can help reduce memory overhead, it does not universally improve performance, and selective application where memory copies dominate may be more effective.

Comparison of Optimization Approaches

To evaluate the effectiveness of the individual optimizations applied to the original code, we compare execution times across different stages of optimization. Figure 3 visualizes the impact of each approach for $N = 400$ and $N = 1000$. The total execution times for each optimization stage are summarized in Table 2, along with the percentage improvement relative to the original implementation.

Optimization Stage	**Time ($N = 1000$)**	**Speedup (%)**
Original (Baseline)	15.414 s	0%
Optimized `W`	13.242 s	14.1%
Optimized `getPairwiseSeparations`	11.047 s	28.3%
Optimized `gradW`	13.537 s	12.2%
All Three Optimizations	8.774 s	43.1%
Memoryview + All Optimized	10.934 s	29.1%

Table 2: Execution time and speedup for different optimization stages ($N = 1000$).

The results for $N = 1000$, indicate that optimizing `getPairwiseSeparations` led to the highest individual speedup of 28.3%, significantly reducing execution time. Optimizing `W` and `gradW` showed more moderate improvements of 14.1% and 12.2%, respectively. The largest gain was observed when applying all three optimizations together, achieving a total speedup of **43.1%**, demonstrating that the optimizations were complementary but not additive.

For $N = 400$, only the fully optimized versions were tested, resulting in performance improvements of 74.8% (all optimizations) and 76.9% (memoryview + all optimizations), both significantly increasing performance compared to the original execution time. Interestingly for this smaller problem size the optimized methods were compatible with memoryview, different to $N = 1000$, where the combination lead to an increase in processing time.

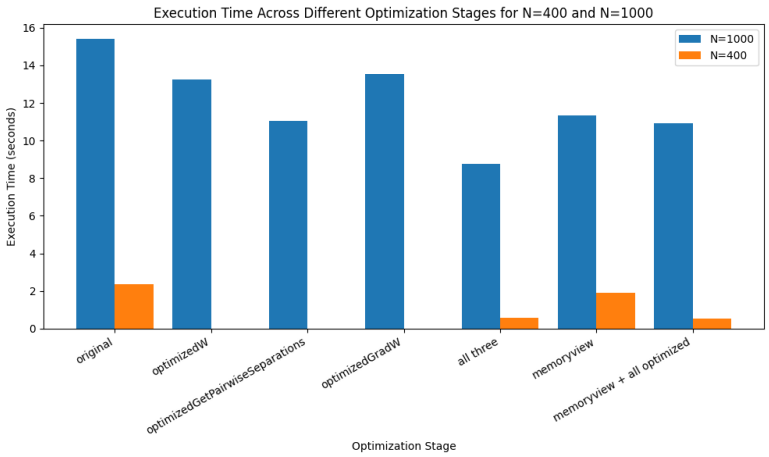


Figure 3: Comparison of different optimization approaches from module 2

Module 2: Optimizing Memory

Following the optimization of CPU execution time, we also directly targeted the optimization of memory use with different data structures. While this did not show significant improvement for the overall memory usage of the program, it did significantly decrease the usage of temporary memory allocations that caused bottlenecks within the code.

Optimizing functions

We focused on optimizing functions that showed the highest increase in memory usage when profiling the code using memory profiler. Four functions were optimized using in place operations. By refactoring the functions to include numpy operations such as `add, sqrt, divide, square, subtract.outer, power, multiply, and negative`, it prevented the creation of temporary arrays in memory. The refactored functions were `W`, `gradW`, `getPairwiseSeparations`, and `getDensity`.

Results of the memory optimizations

As illustrated in Figure 4, the optimizations substantially reduced the maximum memory allocations within the `getDensity` and `W` functions. However, the overall impact on total memory usage was modest, limited to a 6% reduction in memory allocations within the main function. Although using in-place operations to optimize memory usage only yielded moderate gains in total memory usage, they contributed to the execution time improvements demonstrated previously.

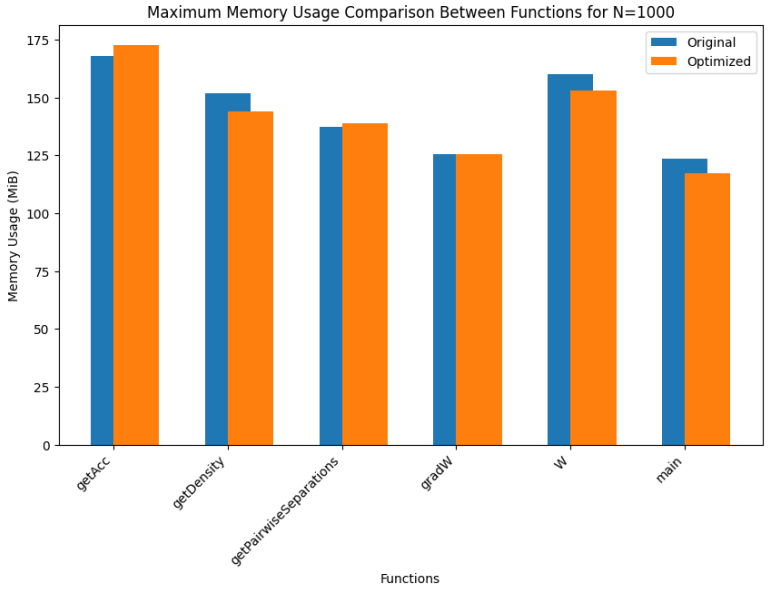


Figure 4: Comparison of memory optimizations

Module 3: GPU Optimization

A critical characteristic of SPH simulations is the inherent data-parallel nature of many computational tasks, such as the calculation of pairwise separations and the evaluation of smoothing kernels. Typically, computations associated with individual particles are independent of each other, aside from aggregation steps (e.g., summations). This intrinsic parallelism aligns precisely with the computational strengths of GPUs. In our implementation, we leveraged CuPy as a direct replacement for NumPy, effectively enabling these data-parallel operations, including matrix calculations and kernel evaluations, to utilize the GPU's extensive parallel architecture, comprising thousands of cores.

Measurement

All measurements were performed in a Google Colab environment. This setup provides access to a remote GPU, but it also has potential variability in system load and resource sharing. To reduce the effect of this, multiple tests were run for each configuration, number of particles N, to compute average execution times and standard deviations.

N	CPU Avg (s)	CPU Std (s)	GPU Avg (s)	GPU Std (s)
400	3.1636	0.4808	0.7068	0.0182
1000	20.7164	0.1592	2.8838	0.0127
1600	73.6636	0.1852	7.0074	0.0055
2200	155.0975	0.2223	13.0562	0.0096

Table 3: CPU and GPU execution times (average and standard deviation) for different particle counts N.

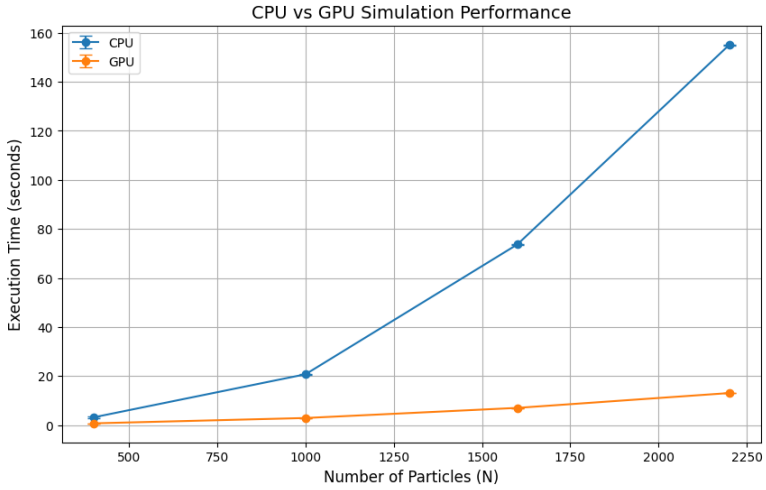


Figure 5: Comparison of simulation performance on GPU vs. CPU with measurement error std. deviations

Performance Results

1. CPU Execution Time: Increases rapidly with N, showing a near-quadratic trend for larger particle counts. For example, the CPU time goes from 3.16 seconds at $N = 400$ to 159 seconds at $N = 2200$.

2. GPU Execution Time: Grows more slowly, e.g., from 0.71 seconds at $N = 400$ to 13.06 seconds at $N = 2200$. The GPU consistently outperforms the CPU, and the speedup factor increases with N. At lower N, GPU overhead (kernel launches, data transfers) is more pronounced; at higher N, the GPU's parallelism dominates, leading to more substantial speedups.

Measurement Error and Variance

By running multiple trials per dataset size, we captured average runtimes and standard deviations, with the GPU times generally exhibiting lower variance (on the order of hundredths of a second). The CPU runs occasionally showed higher standard deviations, reflecting Python's runtime fluctuations and potential CPU load variations. However, these error bars remain relatively small compared to the observed performance gap between CPU and GPU.

GPU Results

Our results from this optimization technique show that, while CPU implementations may be adequate for smaller-scale systems, GPU acceleration becomes essential in realistic SPH applications involving thousands or millions of particles, as it significantly reduces computational time and ensures feasible simulation runtimes.

Module 3: Cython Optimizations

We predicted the Cython optimizations to be quite successful due to the repetition and Numpy arrays utilized in the original code. The main challenge for this implementation was the lack of support for some NumPy list operations in Cython memoryview which required extra time for restructuring those sections of code. We first used Cython to profile the original code and found that all methods had significant Python interactions that could benefit from compiling.

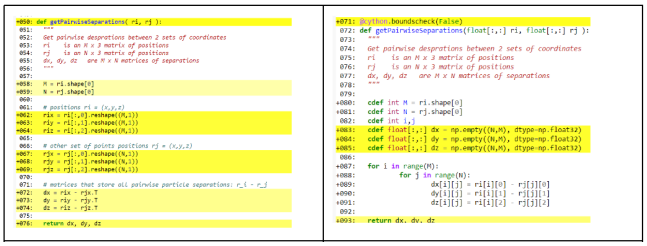


Figure 6: Cython profiling comparison of `getPairwiseSeparations`

Originally, we took a simplistic approach, simply statically typing each variable and converting NumPy arrays to memoryview, and making the minimal necessary modifications to use operations compatible with memoryview. Since NumPy elementwise operations cannot be used with memoryview, it was necessary to explicitly apply the operations to each element using loops. We found that this led to approximately 80-100% slower execution time than the original code. We attributed this behavior to losing the benefits of NumPy vectorization. To mitigate this slowdown, we took a closer look at the design choices in the original code.

We found several auxiliary data structures that were used to enable element-wise NumPy array operations. For example, to combine two vectors into an array, as is done in `getPairwiseSeparations`, one vector might be transposed to allow NumPy to automatically work along both axes. This preparation was no longer necessary if we applied the operations directly to the data in memoryview, so we were able to eliminate these structures. Because of how we applied operations, we were also able to Cythonize the code without bounds checking, to eliminate that overhead. Finally, certain computational operations were optimized to reduce overhead. For example, exponential operations such as `x**2` were replaced by direct multiplication (`x*x`). Additionally, NumPy-specific vectorized operations, which are no longer beneficial when applied to elements directly rather than to entire arrays, were systematically replaced by efficient C-level operations.

The refactored code has significantly fewer Python interactions than the original code 6. With these changes, we achieved a 20% faster execution compared to the original code.

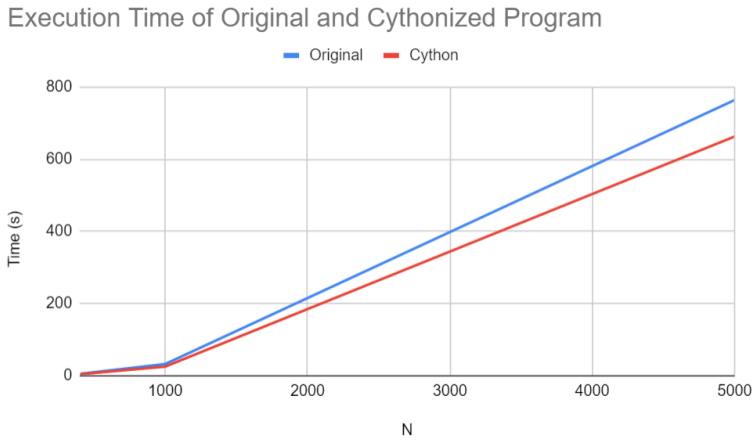


Figure 7: Execution time comparison of original and Cythonized code

Refactoring the code in this way required a thorough exploration of each decision Mocz made, as well as

supplemental research into NumPy and Cython documentation. We found this process to be an extremely valuable experience. It emphasized for us the importance of considering not just the ways in which to improve upon an original design, but the reasons the designer may have made the choices they did in the first place. NumPy operations are less efficient than C operations when directly applied to individual elements in an array, and didn't make sense for our Cythonized version. However, we could appreciate that they were more efficient than our approach would have been when applied to entire NumPy arrays.

Additionally, preliminary experiments utilizing Cython's parallelism capabilities through the `prange` construct were conducted. However, comprehensive results could not be fully developed within the available timeframe.

Module 4: Multiprocessing optimizations

The initial SPH (Smoothed-Particle Hydrodynamics) solver computes pairwise separations and smoothing-kernel contributions by iterating over all particles within a single Python process. Because modern CPUs usually have multiple cores, our aim was to parallelize these loops using Python's `multiprocessing` library. The general strategy in all parallel versions is to replace the direct NumPy calculations inside `W`, `gradW`, and `getPairwiseSeparations` with parallel tasks submitted to a `multiprocessing.Pool`. After computing partial results in each process, the results are gathered back in the main process.

sph_parallel

In `sph_parallel.py`, we created a global `multiprocessing.Pool()` object and replaced certain loops with calls to `POOL.starmap`. For instance, to compute the smoothing function `W` on arrays of distances r , we pass a list of (r_i, h) pairs to `starmap`, then convert the returned list into a NumPy array. The function `calcw` is a small helper that evaluates the Gaussian smoothing for a single distance. Similarly, `calcn` handles the gradient factor. Although this splits up the workload among processes, each call to `starmap` must serialize the data to child processes, and the code still creates large intermediate arrays, which leads to overhead that overshadowed any potential speedup.

sph_parallel_shared_memory

In the next variant, `sph_parallel_shared_memory.py`, we attempt to reuse the POOL throughout the simulation (rather than creating and destroying pools repeatedly). In this version we still call `POOL.starmap` for each major loop, but in a slightly more centralized manner. The logic for pairwise separations and the smoothing kernels remains conceptually the same, but by maintaining a single `Pool` across the entire run, we hoped to reduce overhead from frequent pool setup or process forking. In practice, although the pool is kept open across the entire simulation, the overall overhead (data transfers and partial-result gathering) was still large enough that the observed performance did not exceed the baseline single-process code.

sph_parallel_shared_memory_batched

In `sph_parallel_shared_memory_batched.py`, we use the `chunksize` parameter in `starmap`. Instead of letting `Pool` choose how to batch tasks, we explicitly set various chunk sizes in hopes that a careful balance might reduce per-task overhead. The code is otherwise very similar to `sph_parallel_shared_memory.py`: it uses a single persistent pool, but when distributing tasks such as distance evaluations or gradient calculations, the chunk size is fixed to a user-provided integer. Despite testing a range of batch sizes (from 1 to 1000), none yielded a net speedup over the single-process approach. The overhead of moving data into tasks and collecting results continued to be the dominant factor, especially given that the per-particle kernel computations remain relatively light.

Multiprocessing Results

We ran all multiprocessing variants on an AMD Ryzen 5 5600U (12×2.301,GHz). Figure 8 shows that none of the parallel codes outperformed the original `sph.py`. To conclude, while parallelization might benefit SPH computations in contexts with more computationally expensive kernels or a lower communication overhead, the Python specific overhead and data movement costs in our implementations prevented any processing speed gain.

Module 4: Dask

Since our computational task involves small, dense matrix operations, such as pairwise separations, smoothing kernel evaluations, and summations, we examined whether the Dask framework could enhance performance.

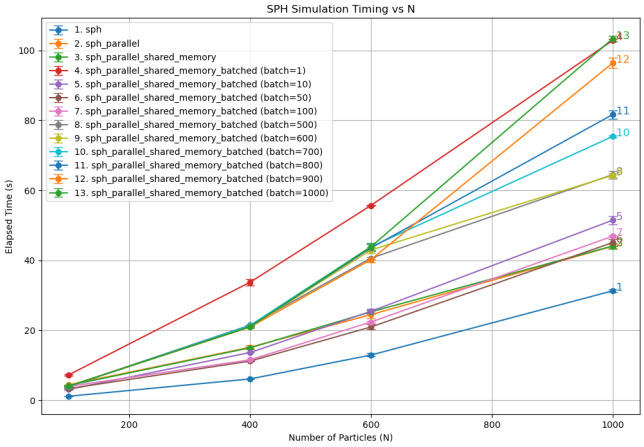


Figure 8: Comparison execution speed mutiprocessing attempts and original implementation

For us the appeal with Dask was its ability to parallelize tasks and optimize memory use through Dask Arrays, Dask Delayed, and Dask Futures. However, our analysis indicates that Dask does not offer a clear speedup, as reflected in execution time results shown in Figure 11.

Dask Arrays, designed for large, out-of-core computations, show no advantage for our relatively small problem sizes (N=1000, N=10000), where NumPy already manages memory efficiently. Frequent synchronization and iterative updates to key variables (pos, vel, acc) introduce overhead, slowing execution instead of accelerating it. Dask Delayed, though suited for independent computations like pairwise separations, suffers from excessive scheduling overhead due to frequent dependency resolution in our iterative loop (Nt=300), reducing potential gains.

Surprisingly, for N=10000, Dask Arrays significantly outperformed other methods, including optimized NumPy. While we initially expected task scheduling overhead to degrade performance, Dask may have parallelized operations more effectively across CPU cores or optimized computation order via lazy execution. However, this improvement was not observed with Dask Delayed, and the exact cause remains unclear. We also attempted to use `dask.map_overlap` to improve performance further, but it did not work as expected and introduced numerous errors in our code, making it difficult to integrate effectively.

Dask Futures, designed for distributed workloads, proved ineffective in our single-node environment. The additional overhead of task distribution and dependency tracking resulted in the worst execution times see 11, demonstrating its unsuitability for this problem.

Overall, Dask-based parallelization is not beneficial for small-scale, memory-efficient simulations like ours. Instead, optimized NumPy implementations leveraging vectorization, memory locality, and minimal task management overhead provide the best performance.

Discussion

Our results demonstrate that strategic optimizations in data layout, vectorization, and efficient processor use substantially improve computational efficiency. Profiling highlighted the critical computational bottlenecks, allowing targeted optimization of key functions. Vectorization and improved data structures provided immediate performance enhancements, GPU optimization provided the most significant speed up, and parallelization approaches yielded negative results.

Challenges and Limitations

Implementing effective parallelization using Dask posed considerable challenges, despite investigating various configurations such as Dask Arrays, Dask Delayed, and Dask Futures. Our use of multicore systems as opposed to a real distributed system was counterproductive to Dask's design and limited its benefits. Additionally, high serialization overhead and complex task management hindered measurable improvements at the tested problem scales. Implementation of parallelism with Python multiprocessing raised a similar issue. At the problem size that our systems could handle, the benefits of multiprocessing techniques were outweighed by the overhead they introduced. Although both Dask and multiprocessing showed limited success for the current scale, they could offer greater potential for larger-scale simulations ($N \geq 10^4$), especially if combined with more advanced architecture.

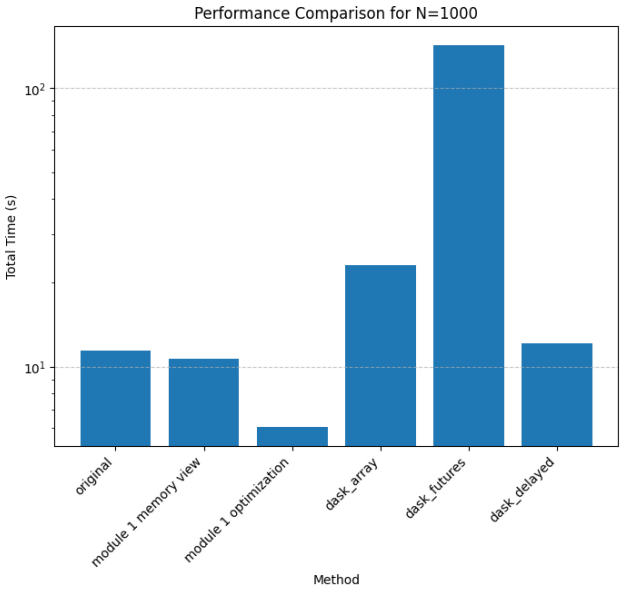


Figure 9: Performance Comparison for $N = 1000$

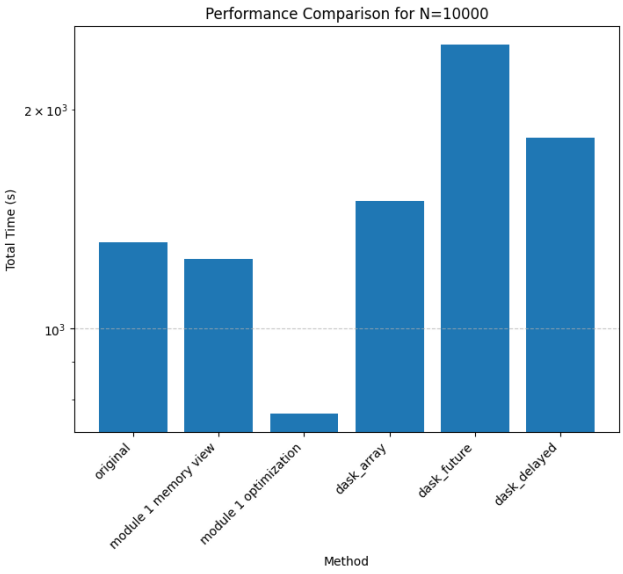


Figure 10: Performance Comparison for $N = 10000$

Figure 11: Comparison of execution times across different optimization methods for $N = 1000$ and $N = 10000$.

Compiling the code with Cython required us to significantly refactor several methods that relied on elementwise NumPy array operations, which was a more time-intensive process than expected. This process highlighted for us that the smallest details, such as the most efficient version of an operation, can not only make a major difference in timing but can also depend on the rest of the program's implementation details. Despite the additional time the process took, we found it to be influential in making us more mindful and thorough programmers. Unfortunately, due to time constraints, additional code restructuring was limited, preventing extensive re-engineering needed to fully exploit multiprocessing and Dask capabilities. Moreover, the program which we optimized is a somewhat simplified framework, which limits the direct applicability of our findings to realistic astrophysical simulations. For example, the Gaussian kernel function (W) is simpler and smoother than kernels often used in astrophysics, such as cubic spline kernels. Furthermore, due to technological limitations, the tested particle counts remain significantly below typical realistic astrophysical applications.

We observed great returns with relatively simple optimizations, supporting the principle that substantial performance gains can be achieved with moderate optimization effort, whereas advanced approaches often provide smaller incremental improvements, if not none, considering multiprocessing and Dask. Furthermore, optimization techniques did not always combine additively and occasionally negated each other's effectiveness, emphasizing the importance of selecting complementary methods carefully.

Ultimately, balancing code readability and maintainability with performance remains essential, and optimization strategies must be selected according to the specific demands and scale of the problem being addressed.

Conclusion

In this project, targeted optimizations significantly enhanced the computational efficiency of the SPH algorithm by addressing critical performance bottlenecks identified through systematic profiling. Improvements in data layouts, strategic vectorization, and compilation contributed to substantial reductions in execution time, particularly evident when leveraging GPU acceleration. However, trade-offs between complexity, maintainability, and performance were observed, indicating the necessity of carefully balancing these aspects when optimizing computationally intensive algorithms.

Future Work

Future investigations should include testing optimized implementations on physically realistic SPH scenarios, involving variations in physical parameters and larger particle counts ($N \geq 10^4$). Exploring distributed parallelization methods, such as cluster-based implementations using Dask or MPI, could further highlight opportunities for performance gains. Additionally, more sophisticated profiling and optimization frameworks, such as just-in-time compilation via Numba or deeper integration with specialized GPU libraries, should be considered to fully exploit available computational resources.

A Contributions

Phoebe Schwartz: Module 2: memory optimizations, Module 4: Dask Array optimizations
Carlotta Hölzle: Profiling, Report, Slides for presentation, Module 2: vectorization optimization, Module 4: Dask Array, Future, Delayed
Rose Maresh: Module 3 Cython: *profiling, optimizations, analysis*; Module 4 Multiprocessing: *starmap optimization*
Christiane Kobalt: Module 3 GPU: *CuPy implementation, profiling, chapter in report, presentation*; Module 4 Multiprocessing: *shared memory optimization, batching optimization, profiling, chapter in report, presentation*

A.1 References

"Cython 3.1.0b0 Documentation." Read The Docs, cython.readthedocs.io/en/latest/. Accessed 14 Mar. 2025.

Mocz, Philip. "Create Your Own Smoothed-Particle Hydrodynamics Simulation (with Python)." Medium, Medium, 19 Sept. 2020, philip-mocz.medium.com/create-your-own-smoothed-particle-hydrodynamics-simulation-with-python-76e1cec505f1.

"Multiprocessing - Process-Based Parallelism." Python Documentation, docs.python.org/3/library/multiprocessing.html. Accessed 14 Mar. 2025.

"NumPy Documentation - NumPy v2.3.Dev0 Manual." Numpy.Org, numpy.org/devdocs/. Accessed 14 Mar. 2025.